



The 40th Annual AAAI Conference on  
Artificial Intelligence  
JANUARY 20 – JANUARY 27, 2026 |  
SINGAPORE

# Uncovering Pretraining Code in LLMs: A Syntax-Aware Attribution Approach

Yuanheng Li<sup>1\*</sup>, Zhuoyang Chen<sup>1\*</sup>, Xiaoyun Liu<sup>1</sup>, Yuhao Wang<sup>1</sup>, Mingwei Liu<sup>2</sup>,  
Yang Shi<sup>1</sup>, Kaifeng Huang<sup>1†</sup>, Shengjie Zhao<sup>1</sup>

1: School of Computer Science and Technology, Tongji University

2: School of Software Engineering, Sun Yat-sen University

\*: Equal contribution, †: Corresponding author



同济大学  
TONGJI UNIVERSITY



中山大学  
SUN YAT-SEN UNIVERSITY

## Contacts

[kaifengh@tongji.edu.cn](mailto:kaifengh@tongji.edu.cn)

kaifeng-h (Wechat)

## Background

- LLM coding capabilities are derived from open-source software, which is subject to strict licensing obligations. Historical GPL litigations demonstrate the consequences of license violations.
- Recent lawsuits highlight legal and ethical risks in using copyrighted text and code for model training.
- Therefore, it underscores the need for transparency, accountability, and protection of human creativity in LLM development.

## Intuition

- While methods like Min-K apply to code, strict syntax rules introduce predictable tokens that encode language constraints rather than authorship.
- Including such inevitable tokens weakens membership inference signals in code-related LLMs.



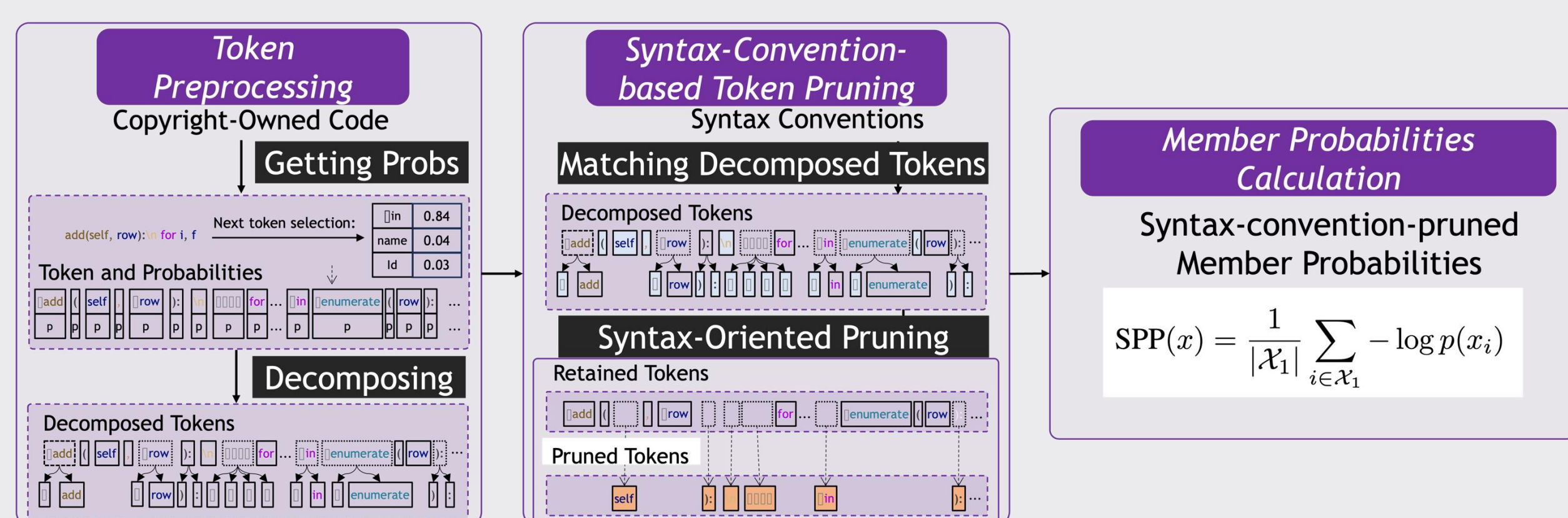
## SynPrune

We study Python syntax conventions and identify tokens that are mandatory and highly predictable, deriving 47 Python syntax conventions.

| Category       | Syntax Nodes                                                      | Example Syntax Node            | Conditional Token                      | Consequent Token |
|----------------|-------------------------------------------------------------------|--------------------------------|----------------------------------------|------------------|
| Data Model     | List, Slice, Dict, Set, String, Bytes, Object, Tuple              | List                           | '['                                    | ']'              |
|                |                                                                   | Dict                           | '{'                                    | '}'              |
|                |                                                                   | Tuple                          | '('                                    | ')'              |
| Expression     | call, lambda, conditional, comprehension, call Chained Comparison |                                | <i>identifier</i>                      | <i>'</i>         |
|                |                                                                   |                                | <i>identifier(identifier)</i>          | <i>'</i>         |
|                |                                                                   | conditional chained comparison | <i>expr if cond</i>                    | <i>else</i>      |
|                |                                                                   |                                | <i>expr comp_op expr</i>               | <i>comp_op</i>   |
| Single Stmts   | import, assert, global                                            | import                         | <i>import module</i>                   | <i>as</i>        |
|                |                                                                   |                                | <i>from module</i>                     | <i>import</i>    |
|                |                                                                   | for                            | <i>for target_list</i>                 | <i>in</i>        |
|                |                                                                   |                                | <i>for target_list in starred_list</i> | <i>in</i>        |
| Compound Stmts | if, for, try, with, class, function, while, match                 |                                | <i>if assignment_expression</i>        | <i>if</i>        |
|                |                                                                   |                                | <i>try: suite except expression</i>    | <i>as</i>        |
|                |                                                                   |                                | <i>def funcname(identifier)</i>        | <i>def</i>       |
|                |                                                                   |                                | <i>with with_item</i>                  | <i>with</i>      |
|                |                                                                   |                                | <i>while condition</i>                 | <i>while</i>     |
|                |                                                                   |                                | <i>match subject</i>                   | <i>match</i>     |

Table 1: List of the Collected Syntax Conventions ([SP], BR, and [IND] denote the space, line break, and indentation, resp.)

We propose SynPrune, a syntax-pruned MIA that removes syntax-determined tokens before membership inference.



## Member Inference Attack (MIA)

- MIAs were used to detect training samples in machine learning. Early MIAs relied on shadow models that mimic the target model’s architecture and training process.
- In language models, MIAs reveal both verbatim and non-verbatim memorization, with the latter being significantly harder to detect.
- A recent work (Shi et al., ICLR 2024) proposes Min-K based on the hypothesis that non-member samples tend to contain a small number of low-probability (outlier) tokens, whereas member samples are less likely to exhibit such tokens.

## Evaluation

- We construct an Python MIA benchmark with confirmed members and non-members for LLM evaluation.
- SynPrune consistently outperforms all baselines across balanced and imbalanced member/non-member ratios, achieving an average AUROC improvement of **15.4%** over prior methods.
- Analyses show strong robustness, meaningful contributions from all syntax conventions, generalizable thresholds across models.

| Ratio | Method   | Pythia 2.8B | GPT-Neo 2.7B | StableLM-Alpha 3B | GPT-J 6B |
|-------|----------|-------------|--------------|-------------------|----------|
| 1:1   | LOSS     | 38.4        | 43.7         | 33.3              | 43.3     |
|       | ZLIB     | 33.2        | 35.5         | 32.0              | 35.7     |
|       | MIN-K    | 38.8        | 41.8         | 34.2              | 41.1     |
|       | DC-PDD   | 50.1        | 41.2         | 43.7              | 41.3     |
|       | SYNPRUNE | 61.3        | 59.7         | 61.3              | 59.7     |
| 1:5   | LOSS     | 40.3        | 45.3         | 33.9              | 44.8     |
|       | ZLIB     | 34.0        | 36.2         | 32.5              | 36.3     |
|       | MIN-K    | 40.2        | 43.3         | 34.9              | 42.4     |
|       | DC-PDD   | 51.8        | 44.0         | 44.6              | 43.2     |
|       | SYNPRUNE | 61.2        | 59.4         | 61.1              | 61.5     |
| 5:1   | LOSS     | 40.0        | 43.8         | 34.9              | 43.6     |
|       | ZLIB     | 33.1        | 35.0         | 31.9              | 35.2     |
|       | MIN-K    | 39.9        | 42.2         | 35.1              | 41.6     |
|       | DC-PDD   | 49.4        | 40.9         | 44.1              | 39.6     |
|       | SYNPRUNE | 62.0        | 60.7         | 62.0              | 63.1     |

Table 4: AUROC (%) of Different Methods under Varying Member-to-non-member Ratios

## Conclusion

We propose SynPrune, a syntax-pruned MIA designed specifically for code-based membership inference. By excluding syntax-constrained tokens, SynPrune isolates semantically meaningful signals of memorization.

### Future Work

- Language Extension
- LLM Extension
- Benchmark Extension
- Uniform cross-language syntax convention design

## Acknowledgement

This work was supported in part by the National Key R\&D Program of China (Grant No. 2023YFC3806000 and 2023YFC3806002), in part by the National Natural Science Foundation of China (Grant No. 62402342, 62402113), in part by Shanghai Municipal Science and Technology Major Project No. 2021SHZDZX0100, and in part by Shanghai Sailing Program (No. 24YF2749500).